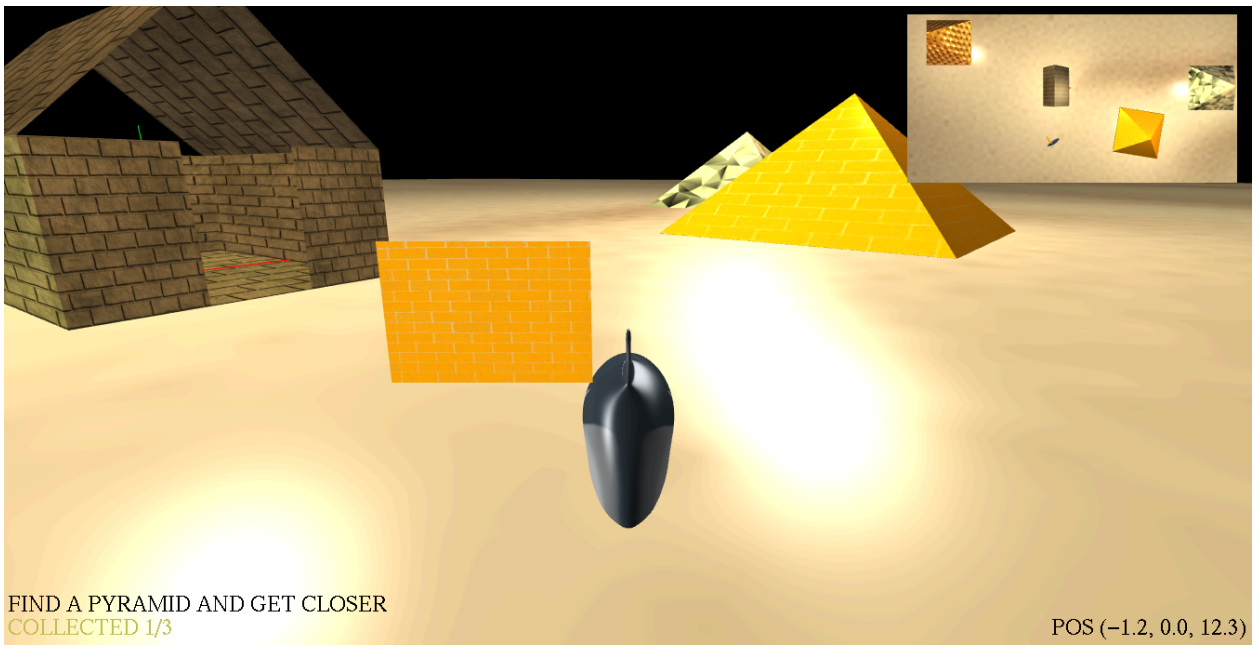
1. Givin Yang, CSC-165 Section-02, A2 – Dolphin Archaeologist – Spring 2026
2. Screenshot:



   I included a screenshot that shows a typical scene from my game with the dolphin, at least one pyramid, the home, the sandy ground plane, hud guide, a picture taken, and the overhead viewport visible.
3. Keyboard and gamepad controls for moving the dolphin
   The dolphin is the avatar and stays on the sandy ground plane at y=0, except for jumping. The keyboard controls are W to move forward, S to move backward, A to turn left using global yaw, and D to turn right using global yaw. The left arrow key strafes left, and the right arrow key strafes right. Left Shift jumps. P takes a photo. Space places the photos on the home wall when you are close enough to home. T toggles the XYZ axes on and off. I also added a help guide toggle, so H will pop up the full controls on screen, and pressing H again will hide them. On the gamepad, the left stick Y moves forward and backward, and the left stick X strafes left and right. The right stick X turns left and right using global yaw. The Y button jumps. The X button takes a photo. The A button places the photos on the home wall. The B button toggles the XYZ axes on and off. I also added a help guide toggle for the controller, and pressing the right stick down will pop up the full controls on screen, and pressing it again will hide it.
4. Orbit controller controls and overhead viewport controls
   The main viewport uses the orbit camera controller and always follows the dolphin as it moves and turns. The keyboard orbit camera controls are Q to orbit left around the dolphin and E to orbit right around the dolphin. The up arrow and down arrow change the camera elevation angle. Z zooms out and X zooms in. The gamepad orbit camera controls are LB to orbit left, RB to orbit right, right stick Y changes elevation angle, Back zooms in, and Start zooms out. The overhead viewport shows the same world from above and has its own pan and zoom controls. The keyboard overhead camera controls are I to pan forward, K to pan backward, J to pan left, and L to pan right. U zooms in by

lowering the overhead camera, and O zooms out by raising the overhead camera. The gamepad overhead camera controls use the D pad to pan, and the triggers to zoom. LT zooms in by lowering the overhead camera and RT zooms out by raising the overhead camera.

5. Node controllers and how they are used
   Two-node controllers are used. The first is RotationController, which is the built-in TAGE rotation controller. When a pyramid is photographed, it becomes a target of this controller and starts rotating in place, so you can tell it has been activated. The second is BobbingController, which is the custom controller I added to TAGE under tage.nodeControllers. It makes a target bob up and down using a sine wave on local Y translation. I use it on the photo objects while they are attached to the dolphin so the collected photos feel active. After the photos are placed on the home wall, the bobbing controller is disabled so the wall photos stay still.
6. A list of all changes made to the TAGE engine
   I added CameraOrbit3D.java in the tage package. It provides orbit, elevation, and zoom controls around a target GameObject and keeps the camera in the same relative position while the dolphin moves and turns. I added BobbingController.java in the tage.nodeControllers. It extends NodeController and applies sine wave bobbing to targets. I changed GameObject.java by adding globalYaw(float radians) so yaw can be applied in world space around the world Y axis. I changed HUDmanager.java by adding support for HUD3 and HUD4 with setHUD3 and setHUD4, so I could guide the player better and also show extra HUD text. I also added a help guide toggle in my game so the guide does not always list controls, and you can press H on the keyboard or press the right stick on the controller to show and hide the full control list. I updated and regenerated the Javadocs, and the updated Javadocs folder is included in the submission.
7. Requirements that were not able to get working
   All requirements were completed and work as intended. I ran many tests, and it is stable.
8. Assets used and legal usage
   Models and shapes
   dolphinHighPoly.obj is from the TAGE helloDolphin example or course provided assets
   ManualQuad.java is my custom shape used for the home walls and the photo quads
   ManualPyramid.java is my custom shape used for the pyramids
   Plane is the built-in TAGE plane shape for the ground

   Textures
   Dolphin_HighPolyUV.jpg is from the TAGE helloDolphin example or course provided assets
   brick1.jpg is from the TAGE helloDolphin example or course provided assets
   sandbyme.jpg is made for this project
   pyramidbyme_512x512.jpg is made for this project

   pyramid2_512x512.jpg is from Freepik
   Resource name is Abstract background with 3d polygons
   Author is s.salvador

3d-geometric-texture-copper_512x512.jpg is from Freepik
Resource name is 3d geometric texture in copper
Author is rawpixel.com
Freepik allows using free resources, and if you are using free resources, you must give credit using an attribution line like Designed by Freepik. I included the sources below as proof, and I am giving credit for the two textures above.

Freepik resource page for pyramid2 texture
https://www.freepik.com/free-vector/abstract-background-with-3d-polygons_5563871.htm

Freepik resource page for copper texture
https://www.freepik.com/free-photo/3d-geometric-texture-copper_3472070.htm

Freepik how to use free images
https://support.freepik.com/s/article/How-can-I-use-free-images?language=en_US

Freepik attribution guide
https://www.freepik.com/ai/docs/attribution-how-when-and-where

9. Machine in RVR-5029 verified
   I verified the program runs correctly on the RVR-5029 machine called Ecs-doom. It is the machine furthest away from the entrance door, where it was tested and worked.
10. Extra changes and gameplay details
    One extra change I added is how the collected photos bob while they are attached to the dolphin. I did not want all the photos to move the same way at the same time, because it looks too robotic and makes it hard to tell them apart. So I made the bobbing controller track each photo separately and give each one its own phase offset and timer. That means when you take the first picture, it will bob in one pattern, and when you take the second picture, it will bob in a different pattern, so they are not in sync. I also added a clean enable and disable flow for the bobbing photos. While the photos are following the dolphin, bobbing is enabled, so the photos look active, and you can clearly see which ones you collected. When you return home and place the photos on the wall, I disable bobbing so the three pictures stay still and look like real wall decorations. For the lose and win situations, I kept the collision punishment. If you get too close to a pyramid and collide with it, you crash and lose the game. If you successfully take all three photos and place them on the wall at home, you win. I also added short event messages so you get quick feedback for actions like taking a photo, not being close enough, crashing, and placing the photos.